

Linked List - 30 LeetCode Questions with Explanations

LINKED LIST — 30 LeetCode Questions (Easy → Hard) | Java

With Teaching Explanations for Each Algorithm

Common Node class (used throughout):

```
class ListNode {
    int val;
    ListNode next;
    ListNode() {}
    ListNode(int val) { this.val = val; }
    ListNode(int val, ListNode next) { this.val = val; this.next =
next; }
}
```

🟡 EASY (1-10)

1. Traverse and Print Linked List

Algorithm: Iterative Traversal **Explanation:** The foundational linked list operation — start at head and follow the next pointers one node at a time until you hit null. Every other linked list algorithm builds on this basic “walk the chain” pattern. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function printList(head):
    curr = head
    while curr != null:
        print(curr.val)
        curr = curr.next
```

Java Solution:

```
class Solution {
    public void printList(ListNode head) {
        ListNode curr = head;
        while (curr != null) {
            System.out.print(curr.val + " ");
            curr = curr.next;
        }
    }
}
```

2. Find Length of Linked List

Algorithm: Iterative Counting **Explanation:** Walk the list once, incrementing a counter for every node visited. Unlike arrays, linked lists don't store their size, so you must traverse the whole structure to know how many nodes it has. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function length(head):
    count = 0
    curr = head
    while curr != null:
        count++
        curr = curr.next
    return count
```

Java Solution:

```
class Solution {
    public int length(ListNode head) {
        int count = 0;
        ListNode curr = head;
        while (curr != null) {
            count++;
            curr = curr.next;
        }
        return count;
    }
}
```

3. Search an Element in Linked List

Algorithm: Linear Traversal **Explanation:** Like linear search on an array, walk node by node comparing values against the target. Because linked lists don't support random access (no indexing), this sequential scan is the only way to search — there's no linked-list equivalent of binary search without extra structure. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function search(head, target):
    curr = head
    while curr != null:
        if curr.val == target: return true
        curr = curr.next
    return false
```

Java Solution:

```
class Solution {
    public boolean search(ListNode head, int target) {
        ListNode curr = head;
        while (curr != null) {
            if (curr.val == target) return true;
            curr = curr.next;
        }
        return false;
    }
}
```

4. Insert Node at Head

Algorithm: Pointer Manipulation ($O(1)$ Insertion) **Explanation:** Create the new node, point its next at the current head, then make the new node the head. Because you never need to traverse anything, inserting at the front of a linked list is $O(1)$ — a key advantage over arrays, where inserting at the front requires shifting every element. **Time Complexity:** $O(1)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function insertHead(head, val):
```

```

newNode = new Node(val)
newNode.next = head
return newNode

```

Java Solution:

```

class Solution {
    public ListNode insertHead(ListNode head, int val) {
        ListNode newNode = new ListNode(val);
        newNode.next = head;
        return newNode;
    }
}

```

5. Insert Node at Tail

Algorithm: Traversal + Pointer Manipulation **Explanation:** Unlike inserting at the head, inserting at the tail of a singly linked list requires walking all the way to the last node first (since you can't jump to the end), then attaching the new node there. This highlights a core linked-list trade-off: fast head insertion, slower tail insertion (unless you maintain a tail pointer). **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```

function insertTail(head, val):
    newNode = new Node(val)
    if head == null: return newNode
    curr = head
    while curr.next != null: curr = curr.next
    curr.next = newNode
    return head

```

Java Solution:

```

class Solution {
    public ListNode insertTail(ListNode head, int val) {
        ListNode newNode = new ListNode(val);
        if (head == null) return newNode;
        ListNode curr = head;
        while (curr.next != null) curr = curr.next;
        curr.next = newNode;
        return head;
    }
}

```

6. Delete a Node by Value

Algorithm: Dummy Node Technique **Explanation:** A “dummy” node placed before the real head simplifies edge cases (like deleting the head itself) because you never have to special-case “is this the first node?” — you always look one step ahead from curr to curr.next and unlink it if it matches. This dummy-node trick appears constantly in linked list problems to avoid messy null checks. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```

function deleteNode(head, val):
    dummy = new Node(0); dummy.next = head
    curr = dummy
    while curr.next != null:
        if curr.next.val == val:
            curr.next = curr.next.next
            break

```

```
curr = curr.next
return dummy.next
```

Java Solution:

```
class Solution {
    public ListNode deleteNode(ListNode head, int val) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode curr = dummy;
        while (curr.next != null) {
            if (curr.next.val == val) {
                curr.next = curr.next.next;
                break;
            }
            curr = curr.next;
        }
        return dummy.next;
    }
}
```

7. Delete Node Given Only That Node (LeetCode 237)

Algorithm: Value Copy Trick **Explanation:** Normally deleting a node requires access to the *previous* node so you can skip over the one being removed. Here you're only given the node itself (not the head or its predecessor). The trick: copy the next node's value into the current node, then unlink the next node instead. It looks like you deleted the current node, but you actually deleted its neighbor after cloning its data forward. **Time Complexity:** $O(1)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function deleteNode(node):
    node.val = node.next.val
    node.next = node.next.next
```

Java Solution:

```
class Solution {
    public void deleteNode(ListNode node) {
        node.val = node.next.val;
        node.next = node.next.next;
    }
}
```

8. Reverse a Linked List (LeetCode 206)

Algorithm: Iterative Pointer Reversal **Explanation:** Walk through the list while flipping each node's next pointer to point backward instead of forward. Three pointers are needed: prev (the reversed part built so far), curr (the node being flipped), and a temporary nextTemp to remember what comes next before you overwrite the link. This is one of the most important linked-list patterns and shows up as a building block inside many harder problems (palindrome check, reorder list, reverse in groups). **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function reverseList(head):
    prev = null, curr = head
    while curr != null:
        nextTemp = curr.next
        curr.next = prev
        prev = curr
        curr = nextTemp
```

```
return prev
```

Java Solution:

```
class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null, curr = head;
        while (curr != null) {
            ListNode nextTemp = curr.next;
            curr.next = prev;
            prev = curr;
            curr = nextTemp;
        }
        return prev;
    }
}
```

9. Find Middle of Linked List (LeetCode 876)

Algorithm: Slow-Fast Pointer (Tortoise and Hare) **Explanation:** Move slow one step at a time and fast two steps at a time. By the time fast reaches the end of the list, slow will be exactly at the middle — because it has covered half the distance. This “two speeds” trick avoids the need to know the list’s length in advance and is the foundation for cycle detection and palindrome checking. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function middleNode(head):
    slow = head, fast = head
    while fast != null and fast.next != null:
        slow = slow.next
        fast = fast.next.next
    return slow
```

Java Solution:

```
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
```

10. Merge Two Sorted Linked Lists (LeetCode 21)

Algorithm: Dummy Node + Two Pointer Merge **Explanation:** Similar to the merge step of merge sort: compare the front nodes of both lists, attach the smaller one to the result, and advance that list’s pointer. A dummy head node makes it easy to build the result without special-casing the very first node. Once one list runs out, attach the rest of the other list directly since it’s already sorted. **Time Complexity:** $O(n + m)$ **Space Complexity:** $O(1)$ extra (reuses existing nodes)

Pseudocode:

```
function mergeTwoLists(l1, l2):
    dummy = new Node(0)
    tail = dummy
    while l1 != null and l2 != null:
        if l1.val <= l2.val:
```

```

        tail.next = l1; l1 = l1.next
    else:
        tail.next = l2; l2 = l2.next
    tail = tail.next
    tail.next = (l1 != null) ? l1 : l2
    return dummy.next

```

Java Solution:

```

class Solution {
    public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode tail = dummy;
        while (l1 != null && l2 != null) {
            if (l1.val <= l2.val) {
                tail.next = l1;
                l1 = l1.next;
            } else {
                tail.next = l2;
                l2 = l2.next;
            }
            tail = tail.next;
        }
        tail.next = (l1 != null) ? l1 : l2;
        return dummy.next;
    }
}

```

🔴 MEDIUM (11-20)

11. Detect Cycle in Linked List (LeetCode 141)

Algorithm: Floyd's Cycle Detection (Tortoise and Hare) **Explanation:** Move slow one step and fast two steps at a time. If there's a cycle, the faster pointer will eventually "lap" the slower one and they'll land on the same node — like two runners on a circular track at different speeds. If there's no cycle, fast simply reaches the end (null) first. This achieves O(1) space, unlike a hash-set-based visited-node approach. **Time Complexity:** O(n) **Space Complexity:** O(1)

Pseudocode:

```

function hasCycle(head):
    slow = head, fast = head
    while fast != null and fast.next != null:
        slow = slow.next
        fast = fast.next.next
        if slow == fast: return true
    return false

```

Java Solution:

```

class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
}

```

12. Find Starting Point of Cycle (LeetCode 142)

Algorithm: Floyd's Cycle Detection — Phase 2 **Explanation:** First, find the meeting point using slow/fast pointers exactly like cycle detection. Then comes the clever second phase: reset one pointer to head and advance both pointers one step at a time — they are mathematically guaranteed to meet exactly at the start of the cycle. This works because of the distance relationship between the head, the cycle's start, and the meeting point that falls out of the two-speed traversal. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function detectCycle(head):
    slow = head, fast = head
    while fast != null and fast.next != null:
        slow = slow.next
        fast = fast.next.next
    if slow == fast:
        ptr = head
        while ptr != slow:
            ptr = ptr.next
            slow = slow.next
        return ptr
    return null
```

Java Solution:

```
class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                ListNode ptr = head;
                while (ptr != slow) {
                    ptr = ptr.next;
                    slow = slow.next;
                }
                return ptr;
            }
        }
        return null;
    }
}
```

13. Remove Nth Node From End (LeetCode 19)

Algorithm: Two Pointer (Fast-Slow with a Gap) **Explanation:** Since you don't know the list's length in advance, move a fast pointer n steps ahead first, creating a fixed gap of n nodes between fast and slow. Then advance both together until fast reaches the end — at that point slow sits right before the node to remove. A dummy node handles the edge case of removing the head itself in one pass, avoiding a separate length-counting pass. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function removeNthFromEnd(head, n):
    dummy = new Node(0); dummy.next = head
    fast = dummy, slow = dummy
    for i from 1 to n: fast = fast.next
    while fast.next != null:
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next
    return dummy.next
```

Java Solution:

```
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode fast = dummy, slow = dummy;
        for (int i = 0; i < n; i++) fast = fast.next;
        while (fast.next != null) {
            fast = fast.next;
            slow = slow.next;
        }
        slow.next = slow.next.next;
        return dummy.next;
    }
}
```

14. Palindrome Linked List (LeetCode 234)

Algorithm: Slow-Fast Pointer + In-Place Reversal **Explanation:** Find the middle using slow/fast pointers, reverse the second half in place (reusing the reversal technique from problem 8), then walk the first half and the reversed second half together, comparing values. If every pair matches, it's a palindrome. This combines two earlier patterns to solve the problem in $O(1)$ extra space instead of copying values into an array. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function isPalindrome(head):
    slow = head, fast = head
    while fast != null and fast.next != null:
        slow = slow.next; fast = fast.next.next
    secondHalf = reverse(slow)
    firstHalf = head
    while secondHalf != null:
        if firstHalf.val != secondHalf.val: return false
        firstHalf = firstHalf.next
        secondHalf = secondHalf.next
    return true
```

Java Solution:

```
class Solution {
    public boolean isPalindrome(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        ListNode secondHalf = reverse(slow);
        ListNode firstHalf = head;
        while (secondHalf != null) {
            if (firstHalf.val != secondHalf.val) return false;
            firstHalf = firstHalf.next;
            secondHalf = secondHalf.next;
        }
        return true;
    }

    private ListNode reverse(ListNode head) {
        ListNode prev = null;
        while (head != null) {
            ListNode next = head.next;
            head.next = prev;
            prev = head;
            head = next;
        }
    }
}
```

```
        return prev;
    }
}
```

15. Add Two Numbers (LeetCode 2)

Algorithm: Simulation (Digit-by-Digit Addition) **Explanation:** Each list represents a number with digits stored in reverse order. Simulate elementary-school column addition: walk both lists simultaneously, add corresponding digits plus any carry from the previous step, and create a new node holding the result's ones digit while carrying the tens digit forward. Continue until both lists and any leftover carry are exhausted. **Time Complexity:** $O(\max(m, n))$ **Space Complexity:** $O(\max(m, n))$ for the output list

Pseudocode:

```
function addTwoNumbers(l1, l2):
    dummy = new Node(0)
    curr = dummy, carry = 0
    while l1 != null or l2 != null or carry != 0:
        x = (l1 != null) ? l1.val : 0
        y = (l2 != null) ? l2.val : 0
        sum = x + y + carry
        carry = sum / 10
        curr.next = new Node(sum % 10)
        curr = curr.next
        if l1 != null: l1 = l1.next
        if l2 != null: l2 = l2.next
    return dummy.next
```

Java Solution:

```
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;
        int carry = 0;
        while (l1 != null || l2 != null || carry != 0) {
            int x = (l1 != null) ? l1.val : 0;
            int y = (l2 != null) ? l2.val : 0;
            int sum = x + y + carry;
            carry = sum / 10;
            curr.next = new ListNode(sum % 10);
            curr = curr.next;
            if (l1 != null) l1 = l1.next;
            if (l2 != null) l2 = l2.next;
        }
        return dummy.next;
    }
}
```

16. Intersection of Two Linked Lists (LeetCode 160)

Algorithm: Two Pointer Switch Technique **Explanation:** Two pointers start at the heads of each list. When a pointer reaches the end of its own list, redirect it to the head of the *other* list. Because both pointers together travel the same total distance ($\text{lengthA} + \text{lengthB}$) before potentially meeting, they arrive at the intersection point at exactly the same time — no need to compute list lengths or offsets manually. **Time Complexity:** $O(m + n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function getIntersectionNode(headA, headB):
```

```

a = headA, b = headB
while a != b:
    a = (a == null) ? headB : a.next
    b = (b == null) ? headA : b.next
return a

```

Java Solution:

```

class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode
headB) {
        ListNode a = headA, b = headB;
        while (a != b) {
            a = (a == null) ? headB : a.next;
            b = (b == null) ? headA : b.next;
        }
        return a;
    }
}

```

17. Odd Even Linked List (LeetCode 328)

Algorithm: Two Pointer Partitioning (In-Place) **Explanation:**

Maintain two separate chains as you traverse — one collecting nodes at odd positions, one collecting nodes at even positions — using two pointers odd and even that each skip ahead by one link per step. Once you reach the end, splice the even chain onto the tail of the odd chain. This groups nodes by position without allocating any new nodes. **Time**

Complexity: O(n) **Space Complexity:** O(1)

Pseudocode:

```

function oddEvenList(head):
    if head == null: return head
    odd = head, even = head.next, evenHead = even
    while even != null and even.next != null:
        odd.next = even.next
        odd = odd.next
        even.next = odd.next
        even = even.next
    odd.next = evenHead
    return head

```

Java Solution:

```

class Solution {
    public ListNode oddEvenList(ListNode head) {
        if (head == null) return head;
        ListNode odd = head, even = head.next, evenHead = even;
        while (even != null && even.next != null) {
            odd.next = even.next;
            odd = odd.next;
            even.next = odd.next;
            even = even.next;
        }
        odd.next = evenHead;
        return head;
    }
}

```

18. Swap Nodes in Pairs (LeetCode 24)

Algorithm: Iterative Pointer Manipulation **Explanation:** Process the list two nodes at a time using a prev pointer that always trails just before the current pair. For each pair, rewire the links so the second node comes first, then advance prev to what is now the end of the

swapped pair. A dummy head simplifies swapping the very first pair. This is a controlled, local application of the reversal idea, limited to groups of two. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function swapPairs(head):
    dummy = new Node(0); dummy.next = head
    prev = dummy
    while prev.next != null and prev.next.next != null:
        first = prev.next
        second = first.next
        first.next = second.next
        second.next = first
        prev.next = second
        prev = first
    return dummy.next
```

Java Solution:

```
class Solution {
    public ListNode swapPairs(ListNode head) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode prev = dummy;
        while (prev.next != null && prev.next.next != null) {
            ListNode first = prev.next;
            ListNode second = first.next;
            first.next = second.next;
            second.next = first;
            prev.next = second;
            prev = first;
        }
        return dummy.next;
    }
}
```

19. Rotate List (LeetCode 61)

Algorithm: Circular Linked List Technique **Explanation:** First find the length and connect the tail back to the head, temporarily turning the list into a circle. Compute where the new head should be ($\text{length} - k \% \text{length}$ steps from the old head), walk there, and break the circle at that point. Treating the list as circular for a moment makes it easy to “wrap around” without complex edge-case handling. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function rotateRight(head, k):
    if head == null: return null
    length = 1, tail = head
    while tail.next != null: tail = tail.next; length++
    k = k % length
    if k == 0: return head
    tail.next = head
    stepsToNewHead = length - k
    newTail = head
    for i from 1 to stepsToNewHead-1: newTail = newTail.next
    newHead = newTail.next
    newTail.next = null
    return newHead
```

Java Solution:

```
class Solution {
    public ListNode rotateRight(ListNode head, int k) {
        if (head == null) return null;
```

```

        int length = 1;
        ListNode tail = head;
        while (tail.next != null) {
            tail = tail.next;
            length++;
        }
        k = k % length;
        if (k == 0) return head;
        tail.next = head;
        int stepsToNewHead = length - k;
        ListNode newTail = head;
        for (int i = 1; i < stepsToNewHead; i++) newTail =
newTail.next;
        ListNode newHead = newTail.next;
        newTail.next = null;
        return newHead;
    }
}

```

20. Partition List (LeetCode 86)

Algorithm: Two Dummy Lists Technique **Explanation:** Build two separate lists while traversing the original once: a “before” list for values less than x, and an “after” list for values greater than or equal to x. Each uses its own dummy head to simplify appending. At the end, stitch the “after” list onto the tail of the “before” list. This preserves the relative order within each group, which a naive swap-based partition would not. **Time Complexity:** O(n) **Space Complexity:** O(1) extra (reuses existing nodes)

Pseudocode:

```

function partition(head, x):
    beforeDummy = new Node(0), afterDummy = new Node(0)
    before = beforeDummy, after = afterDummy
    while head != null:
        if head.val < x:
            before.next = head; before = before.next
        else:
            after.next = head; after = after.next
        head = head.next
    after.next = null
    before.next = afterDummy.next
    return beforeDummy.next

```

Java Solution:

```

class Solution {
    public ListNode partition(ListNode head, int x) {
        ListNode beforeDummy = new ListNode(0);
        ListNode afterDummy = new ListNode(0);
        ListNode before = beforeDummy, after = afterDummy;
        while (head != null) {
            if (head.val < x) {
                before.next = head;
                before = before.next;
            } else {
                after.next = head;
                after = after.next;
            }
            head = head.next;
        }
        after.next = null;
        before.next = afterDummy.next;
        return beforeDummy.next;
    }
}

```

🔴 HARD (21-30)

21. Reverse Nodes in K-Group (LeetCode 25)

Algorithm: Recursion + Iterative Segment Reversal **Explanation:** First check whether at least k nodes remain; if not, leave that final partial group untouched. If there are enough nodes, recursively process everything after the current group first, then reverse the current group of k nodes iteratively (same idea as problem 8, but bounded to k nodes) and attach it to the already-processed, reversed remainder. This combines recursion (to handle “the rest of the list”) with iteration (to reverse a fixed-size chunk). **Time Complexity:** $O(n)$ **Space Complexity:** $O(n/k)$ for the recursion stack

Pseudocode:

```
function reverseKGroup(head, k):
    node = head
    count = 0
    while node != null and count < k:
        node = node.next; count++
    if count == k:
        node = reverseKGroup(node, k)
        while count > 0:
            temp = head.next
            head.next = node
            node = head
            head = temp
            count--
        head = node
    return head
```

Java Solution:

```
class Solution {
    public ListNode reverseKGroup(ListNode head, int k) {
        ListNode node = head;
        int count = 0;
        while (node != null && count < k) {
            node = node.next;
            count++;
        }
        if (count == k) {
            node = reverseKGroup(node, k);
            while (count > 0) {
                ListNode temp = head.next;
                head.next = node;
                node = head;
                head = temp;
                count--;
            }
            head = node;
        }
        return head;
    }
}
```

22. Copy List with Random Pointer (LeetCode 138)

Algorithm: HashMap Mapping Technique **Explanation:** Each node has a random pointer that can point anywhere in the list, including forward — so a straightforward single pass can’t set it correctly since the target node might not exist yet. Solve this by first creating a clone of every node and mapping original -> clone in a HashMap. Then, in

a second pass, use the map to wire up each clone's next and random pointers to the *cloned* versions of the corresponding original nodes.

Time Complexity: $O(n)$ **Space Complexity:** $O(n)$

Pseudocode:

```
function copyRandomList(head):
    if head == null: return null
    map = empty hashmap
    curr = head
    while curr != null:
        map[curr] = new Node(curr.val)
        curr = curr.next
    curr = head
    while curr != null:
        map[curr].next = map[curr.next]
        map[curr].random = map[curr.random]
        curr = curr.next
    return map[head]
```

Java Solution:

```
import java.util.HashMap;

class Node {
    int val;
    Node next, random;
    Node(int val) { this.val = val; }
}

class Solution {
    public Node copyRandomList(Node head) {
        if (head == null) return null;
        HashMap<Node, Node> map = new HashMap<>();
        Node curr = head;
        while (curr != null) {
            map.put(curr, new Node(curr.val));
            curr = curr.next;
        }
        curr = head;
        while (curr != null) {
            map.get(curr).next = map.get(curr.next);
            map.get(curr).random = map.get(curr.random);
            curr = curr.next;
        }
        return map.get(head);
    }
}
```

23. Merge K Sorted Lists (LeetCode 23)

Algorithm: Min-Heap (Priority Queue) **Explanation:** Rather than merging the lists two at a time (which is correct but not optimal), put the head node of every list into a min-heap ordered by value.

Repeatedly pop the smallest node, attach it to the result, and push that node's next (if any) back into the heap. The heap always gives you the globally smallest "next candidate" across all k lists in $O(\log k)$ time, making the whole merge much faster than repeated pairwise merging for large k . **Time Complexity:** $O(N \log k)$, where N is total nodes and k is the number of lists **Space Complexity:** $O(k)$ for the heap

Pseudocode:

```
function mergeKLists(lists):
    minHeap = priority queue ordered by node.val
    for each list in lists:
        if list != null: minHeap.add(list)
```

```

dummy = new Node(0), tail = dummy
while minHeap not empty:
    node = minHeap.poll()
    tail.next = node
    tail = tail.next
    if node.next != null: minHeap.add(node.next)
return dummy.next

```

Java Solution:

```

import java.util.PriorityQueue;

class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        PriorityQueue<ListNode> minHeap = new PriorityQueue<>((a, b)
-> a.val - b.val);
        for (ListNode list : lists) {
            if (list != null) minHeap.offer(list);
        }
        ListNode dummy = new ListNode(0);
        ListNode tail = dummy;
        while (!minHeap.isEmpty()) {
            ListNode node = minHeap.poll();
            tail.next = node;
            tail = tail.next;
            if (node.next != null) minHeap.offer(node.next);
        }
        return dummy.next;
    }
}

```

24. Sort List — Merge Sort on Linked List (LeetCode 148)

Algorithm: Merge Sort (Divide and Conquer) **Explanation:** Find the middle using slow/fast pointers, split the list into two halves, recursively sort each half, then merge the two sorted halves back together (reusing the merge logic from problem 10). Merge sort is the natural sorting algorithm for linked lists because splitting only requires pointer manipulation (no shifting like arrays need), and it guarantees $O(n \log n)$ performance regardless of the input's initial order. **Time Complexity:** $O(n \log n)$ **Space Complexity:** $O(\log n)$ for the recursion stack

Pseudocode:

```

function sortList(head):
    if head == null or head.next == null: return head
    mid = findMiddleAndSplit(head)
    left = sortList(head)
    right = sortList(mid)
    return merge(left, right)

```

Java Solution:

```

class Solution {
    public ListNode sortList(ListNode head) {
        if (head == null || head.next == null) return head;
        ListNode slow = head, fast = head, prev = null;
        while (fast != null && fast.next != null) {
            prev = slow;
            slow = slow.next;
            fast = fast.next.next;
        }
        prev.next = null;
        ListNode left = sortList(head);
        ListNode right = sortList(slow);
        return merge(left, right);
    }
}

```

```

    }

    private ListNode merge(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode tail = dummy;
        while (l1 != null && l2 != null) {
            if (l1.val <= l2.val) {
                tail.next = l1; l1 = l1.next;
            } else {
                tail.next = l2; l2 = l2.next;
            }
            tail = tail.next;
        }
        tail.next = (l1 != null) ? l1 : l2;
        return dummy.next;
    }
}

```

25. Flatten a Multilevel Doubly Linked List (LeetCode 430)

Algorithm: Depth-First Search (Recursion) **Explanation:** Some nodes have a child pointer leading to a separate sub-list. Walk the main list, and whenever a child pointer is found, recursively flatten that child list first, then splice the fully flattened child sequence in between the current node and what used to be its next. This is a linked-list version of a DFS/pre-order traversal, treating the child link like an edge to a subtree that must be fully resolved before continuing. **Time Complexity:** $O(n)$ — every node visited once **Space Complexity:** $O(d)$ recursion stack, where d is the nesting depth

Pseudocode:

```

function flatten(head):
    curr = head
    while curr != null:
        if curr.child != null:
            next = curr.next
            child = flatten(curr.child)
            curr.next = child
            child.prev = curr
            curr.child = null
            tail = child
            while tail.next != null: tail = tail.next
            tail.next = next
            if next != null: next.prev = tail
        curr = curr.next
    return head

```

Java Solution:

```

class Node {
    int val;
    Node prev, next, child;
}

class Solution {
    public Node flatten(Node head) {
        Node curr = head;
        while (curr != null) {
            if (curr.child != null) {
                Node next = curr.next;
                Node child = flatten(curr.child);
                curr.next = child;
                child.prev = curr;
                curr.child = null;
                Node tail = child;
                while (tail.next != null) tail = tail.next;
                tail.next = next;
                if (next != null) next.prev = tail;
            }
            curr = curr.next;
        }
        return head;
    }
}

```

```

        while (tail.next != null) tail = tail.next;
        tail.next = next;
        if (next != null) next.prev = tail;
    }
    curr = curr.next;
}
return head;
}
}

```

26. LRU Cache (LeetCode 146) — Doubly Linked List + HashMap

Algorithm: Doubly Linked List + HashMap (Combined Data Structure Design) **Explanation:** An LRU (Least Recently Used) cache needs $O(1)$ access by key *and* $O(1)$ reordering by recency — no single basic data structure gives both. The trick is to combine them: a HashMap gives instant key lookup, while a doubly linked list keeps nodes ordered from most-recently-used (front) to least-recently-used (back). On every access, the accessed node is unlinked and moved to the front. When the cache is full, the node just before the tail (the least recently used) is evicted. This is a cornerstone system-design-style linked-list problem. **Time Complexity:** $O(1)$ for both get and put **Space Complexity:** $O(\text{capacity})$

Pseudocode:

```

class LRUCache:
    init capacity, hashmap, doubly-linked list (dummy head & tail)

    function get(key):
        if key not in map: return -1
        move node to front (most recently used)
        return node.val

    function put(key, value):
        if key in map: update value, move to front
        else:
            if size == capacity: remove least recently used
            (tail.prev)
            insert new node at front

```

Java Solution:

```

import java.util.HashMap;

class LRUCache {
    class Node {
        int key, val;
        Node prev, next;
        Node(int k, int v) { key = k; val = v; }
    }

    private HashMap<Integer, Node> map = new HashMap<>();
    private int capacity;
    private Node head = new Node(0, 0), tail = new Node(0, 0);

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    private void remove(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }
}

```

```

private void insertFront(Node node) {
    node.next = head.next;
    node.prev = head;
    head.next.prev = node;
    head.next = node;
}

public int get(int key) {
    if (!map.containsKey(key)) return -1;
    Node node = map.get(key);
    remove(node);
    insertFront(node);
    return node.val;
}

public void put(int key, int value) {
    if (map.containsKey(key)) {
        remove(map.get(key));
    }
    Node node = new Node(key, value);
    map.put(key, node);
    insertFront(node);
    if (map.size() > capacity) {
        Node lru = tail.prev;
        remove(lru);
        map.remove(lru.key);
    }
}
}

```

27. Reverse Linked List II (LeetCode 92) — Reverse Between Positions

Algorithm: In-Place Sublist Reversal **Explanation:** Walk to just before position left using a dummy node, then repeatedly move the node right after curr to the front of the sublist being reversed. Each iteration effectively pulls the next node out and re-inserts it right after prev, gradually reversing the segment between left and right without touching the rest of the list. This is a bounded version of full-list reversal (problem 8), applied only to a specific window. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```

function reverseBetween(head, left, right):
    dummy = new Node(0); dummy.next = head
    prev = dummy
    for i from 1 to left-1: prev = prev.next
    curr = prev.next
    for i from 1 to right-left:
        nextNode = curr.next
        curr.next = nextNode.next
        nextNode.next = prev.next
        prev.next = nextNode
    return dummy.next

```

Java Solution:

```

class Solution {
    public ListNode reverseBetween(ListNode head, int left, int
right) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode prev = dummy;
        for (int i = 1; i < left; i++) prev = prev.next;
        ListNode curr = prev.next;

```

```

        for (int i = 0; i < right - left; i++) {
            ListNode nextNode = curr.next;
            curr.next = nextNode.next;
            nextNode.next = prev.next;
            prev.next = nextNode;
        }
        return dummy.next;
    }
}

```

28. Remove Duplicates from Sorted List II (LeetCode 82)

Algorithm: Dummy Node + Skip-Duplicates Traversal **Explanation:** Unlike a simpler duplicate-removal problem where one copy is kept, here *every* node with a duplicate value must be removed entirely. Use a prev pointer that only advances when the current node is confirmed unique. Whenever a run of matching values is detected, skip past the entire run and reconnect prev.next directly to whatever comes after it. The dummy node handles the case where the very first nodes are duplicates. **Time Complexity:** O(n) **Space Complexity:** O(1)

Pseudocode:

```

function deleteDuplicates(head):
    dummy = new Node(0); dummy.next = head
    prev = dummy, curr = head
    while curr != null:
        if curr.next != null and curr.val == curr.next.val:
            while curr.next != null and curr.val == curr.next.val:
                curr = curr.next
            prev.next = curr.next
        else:
            prev = prev.next
            curr = curr.next
    return dummy.next

```

Java Solution:

```

class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode prev = dummy, curr = head;
        while (curr != null) {
            if (curr.next != null && curr.val == curr.next.val) {
                while (curr.next != null && curr.val ==
curr.next.val) {
                    curr = curr.next;
                }
                prev.next = curr.next;
            } else {
                prev = prev.next;
            }
            curr = curr.next;
        }
        return dummy.next;
    }
}

```

29. Reorder List (LeetCode 143)

Algorithm: Find Middle + Reverse + Merge (Three-Step Combination) **Explanation:** To interleave a list as $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$, first find the middle (slow/fast pointers) and split the list in two. Reverse the second half so its nodes come out in reverse order.

Finally, merge the first half and the reversed second half by alternating one node from each. This problem is a great teaching example of composing three previously learned linked-list techniques into a single solution. **Time Complexity:** $O(n)$ **Space Complexity:** $O(1)$

Pseudocode:

```
function reorderList(head):
    find middle using slow/fast pointer
    reverse second half
    merge first half and reversed second half alternately
```

Java Solution:

```
class Solution {
    public void reorderList(ListNode head) {
        if (head == null || head.next == null) return;
        ListNode slow = head, fast = head;
        while (fast.next != null && fast.next.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        ListNode second = slow.next;
        slow.next = null;
        ListNode prev = null;
        while (second != null) {
            ListNode next = second.next;
            second.next = prev;
            prev = second;
            second = next;
        }
        ListNode first = head;
        second = prev;
        while (second != null) {
            ListNode t1 = first.next, t2 = second.next;
            first.next = second;
            second.next = t1;
            first = t1;
            second = t2;
        }
    }
}
```

30. Merge In Between Linked Lists (LeetCode 1669)

Algorithm: Pointer Splicing **Explanation:** Walk list1 to find the node just before position a (call it prev) and the node just after position b (call it after). Splice list2 in between them by pointing prev.next to the start of list2, then walk to the end of list2 and point its tail to after. This is a direct application of pointer rewiring — no new nodes are created, and no reversal or merging by comparison is needed, just careful bookkeeping of “where to cut and where to reconnect.” **Time Complexity:** $O(n + m)$, where n is list1’s length up to b and m is list2’s length **Space Complexity:** $O(1)$

Pseudocode:

```
function mergeInBetween(list1, a, b, list2):
    prev = list1
    for i from 1 to a-1: prev = prev.next
    after = prev
    for i from a-1 to b: after = after.next
    after = after.next
    prev.next = list2
    tail = list2
    while tail.next != null: tail = tail.next
    tail.next = after
```

```
return list1
```

Java Solution:

```
class Solution {  
    public ListNode mergeInBetween(ListNode list1, int a, int b,  
    ListNode list2) {  
        ListNode prev = list1;  
        for (int i = 0; i < a - 1; i++) prev = prev.next;  
        ListNode after = prev;  
        for (int i = a - 1; i <= b; i++) after = after.next;  
        prev.next = list2;  
        ListNode tail = list2;  
        while (tail.next != null) tail = tail.next;  
        tail.next = after;  
        return list1;  
    }  
}
```
